

SOCIAL MEDIA APPLICATION USING GRAPH DATABASE

IF3152 ADVANCED DATABASE MANAGEMENT SYSTEM

A MINI PROJECT REPORT

Submitted by

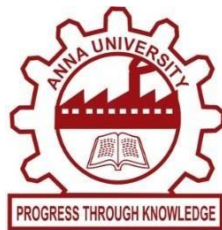
JANANI GOPALAKRISHNAN (2023611026)

SARAN KUMAR S (2023611030)

MASTER OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION TECHNOLOGY

MADRAS INSTITUTE OF TECHNOLOGY CAMPUS,

ANNA UNIVERSITY, CHROMPET,

CHENNAI-600044.

AUG 2023 - DEC 2023

ABSTRACT

In an ever-evolving digital landscape, our project introduces a sophisticated Social Chat application designed to elevate online communication experiences. Leveraging the dynamic capabilities of React.js for the frontend and harnessing the graph-based power of Neo4j Database for backend data management, our application seeks to redefine social interactions within virtual communities. The frontend, powered by React.js, ensures a responsive and user-friendly interface, facilitating seamless navigation and an engaging user experience. Real-time messaging features, interactive notifications, and a visually appealing design contribute to a vibrant and dynamic platform. On the backend, Neo4j Database serves as the foundation for robust data storage and retrieval. Adopting a graph database model, the application organizes user profiles, relationships, and messages in a highly interconnected and efficient structure, enabling optimized queries and scalability. The application's core features include user authentication, group creation, real-time message delivery, and intuitive navigation through social circles. By combining the strengths of React.js and Neo4j Database, our project not only addresses the imperative need for advanced communication tools in the digital age but also delves into the intricacies of integrating diverse technologies for a comprehensive and forward-looking solution. This abstract encapsulates the vision and innovation behind our Social Chat application, showcasing the synergy between React.js and Neo4j Database to create a compelling, efficient, and socially enriching digital space.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTARCT	ii
	LIST OF FIGURES	iv
1	SYSTEM DEVELOPMENT	1
	1.1 Introduction	1
	1.2 Graph Database	1
	1.3 System Design	2
	1.4 Technology Used	2
	1.5 Architecture Diagram	2
	1.6 Module Implementation	3
	1.6.1 Frontend	3
	1.6.2 CSS: Tailwind CSS is used for styling.	11
	1.6.3 Backend	11
	1.6.4 Neo4j Desktop	15
2	OUTPUT AND DISCUSSION	16
3	CONCLUSIONS AND FUTURE	28
	ENHANCEMENTS	
	3.1 Conclusions	28
	3.2 Future Enhancements	28
	REFERENCES	29

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	System Architecture for a Chat Application.	2
1.2	Neo4j Desktop	15
1.3	Server Status for Neo4j Desktop	15
2.1	Index Page	16
2.2	SignUp Page	16
2.3	SignIn Page	17
2.4	Home Page	17
2.5	Registered User's	18
2.6	John sent a message	18
2.7	Janani received a message	19
2.8	Alice sent a message to Lisa	19
2.9	Lisa sent a message to John Doe	20
2.10	Janani sent a message to Lisa	20
2.11	Janani sent a message to John Doe	21
2.12	Janani sent a message to Alice	21
2.13	Lisa sent a message to Alice	22
2.14	Alice sent a message to Janani	22
2.15	All User's	23
2.16	All Messages	23
2.17	Updated an User Email	24
2.18	User's Count	24
2.19	Group and Count	25
2.20	Aggregation with Filtering	25
2.21	New User	26
2.22	Deleting a New User	26
2.23	Deleting a New User along with the Relation	27
2.24	Delete Operation	27

CHAPTER 1

SYSTEM DEVELOPMENT

1.1 INTRODUCTION

In today's interconnected digital landscape, social media has revolutionized the way people communicate and interact. Social media platforms have evolved beyond mere content-sharing spaces to become hubs for real-time conversations and community engagement. Chat applications within these platforms serve as pivotal tools for fostering connections, enabling instant communication, and nurturing relationships among users.

This project aims to delve into the realm of social media chat applications by crafting a dynamic platform that prioritizes the fundamental aspects of user interaction—conversations. Unlike traditional messaging applications, this endeavour focuses on harnessing the capabilities of Neo4j, a graph database technology, to transform the way users engage in dialogue within a social networking environment.

1.2 GRAPH DATABASE

Graph databases, such as Neo4j, represent data in a graph-like structure consisting of nodes, edges, and properties. Unlike traditional relational databases, which organize data in tables and rows, graph databases excel in modeling relationships. Neo4j's graph-based approach allows entities (nodes) to be connected via relationships (edges), enabling a more natural representation of complex networks and intricate connections.

Neo4j's strength lies in its ability to efficiently traverse relationships, enabling rapid querying and manipulation of highly interconnected data. With its flexible data model, Neo4j empowers developers to capture and manage the relationships inherent in social networks—such as friendships, conversations, and group interactions—resulting in a more intuitive and effective representation of social interactions.

Utilizing Neo4j's graph database for a social media chat application offers opportunities to enhance the user experience by enabling smoother and more intuitive communication pathways. The inherent nature of graph databases aligns well with the intricate web of relationships and conversations that define social networking platforms, making Neo4j a powerful foundation for building a dynamic and engaging chat experience.

1.3 SYSTEM DESIGN

To design this Social Media Chat Application using graph database several different components were been used. It is incorporated with graph database to make it more efficient in terms of implementation and user interface.

1.4 TECHNOLOGIES USED

- React.js for Frontend Development.
- Neo4j Database for Backend Data Management.
- WebSocket for Real-time Communication.
- Node.js and Express.js for Backend Server.
- JWT (JSON Web Token) for User Authentication.

1.5 ARCHITECTURE DESIGN

Provides a brief insight into the application's architectural design, focusing on seamless data exchange between frontend and backend components. Rooted in React.js, the frontend architecture emphasizes modularity and dynamic updates through effective state management. Centers around Node.js and Neo4j Database, orchestrating efficient server-side operations, data storage, and real-time communication via WebSocket. Illustrates the data flow between frontend, backend, and the database, offering visual representations of user authentication and real-time messaging sequences in Fig 1.1.

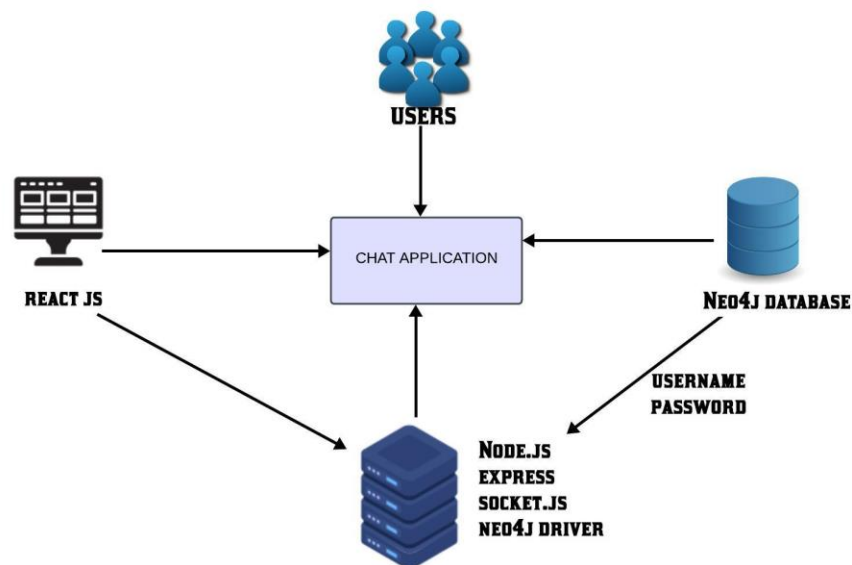


Fig.1.1 System Architecture of a Chat Application.

1.6 MODULE IMPLEMENTATION

The module implementation of the Social Media Chat Application website is as follows.

1.6.1 FRONTEND

Creating Frontend using ReactJS

Package.json

```
{
  "name": "client",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@testing-library/jest-dom": "^5.17.0",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.1",
    "react-scripts": "5.0.1",
    "socket.io-client": "^4.7.2",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  }
}
```

```

},
"devDependencies": {
  "tailwindcss": "^3.3.6"
}
}

```

App.js

```

import './App.css';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Signin from './pages/Signin';
import Signup from './pages/Signup';
import Home from './pages/Home';

function App() {
  return (
    <>
    <Router>
    <Routes>
    <Route path="/" element={<Signin />} />
    <Route path="/Signup" element={<Signup />} />
    <Route path="/Home" element={<Home />} />
    </Routes>
    </Router>
    </>
  );
}

export default App;

```

Home.jsx

```

import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import io from 'socket.io-client';
import ChatItem from './ChatItem';
import man from '../components/images/man.png';
import logo from '../components/images/logo.png';
import out from '../components/images/sign-out.png';
import send from '../components/images/send.png';

function Home() {
  const navigate = useNavigate();
  const [chatData, setChatData] = useState([]);
  const [loading, setLoading] = useState(true);

```



```

const [message, setMessage] = useState("");
const [socket, setSocket] = useState(null);

const handleLogout = (event) => {
  event.preventDefault();
  navigate("/");
};

useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch('http://localhost:5002/api/data');
      if (!response.ok) {
        throw new Error('Failed to fetch data');
      }
      const data = await response.json();
      console.log('Chat data:', data);
      setChatData(data);
      setLoading(false);
    } catch (error) {
      console.error('Error fetching data:', error);
      setLoading(false);
    }
  };

  fetchData();
}, []);

useEffect(() => {
  const socketConnection = io('http://localhost:5002');
  socketConnection.on('connect', () => {
    console.log('Socket connected!');
  });
  socketConnection.on('connect_error', (error) => {
    console.error('Socket connection error:', error);
  });
  socketConnection.on('disconnect', (reason) => {
    console.log('Socket disconnected:', reason);
  });
  setSocket(socketConnection);
  return () => {
    socketConnection.disconnect();
  };
}, []);

useEffect(() => {
  if (socket) {
    socket.on('message', (data) => {
      console.log('Message received from server:', data);
    });
  }
}, [socket]);

```

```

});
}
}, [socket]);

const sendMessage = () => {
  console.log('Message:', message);
  if (socket) {
    socket.emit('message', { text: message });
  }
};

const handleSubmit = (event) => {
  event.preventDefault();
  sendMessage();
  setMessage("");
};

return (
  <div>
    <div className='grid grid-cols-4 '>
      <div className='row-span-1 border border-gray-200 h-screen'>
        <form onSubmit={handleLogout}>
          <div className='flex my-6 ml-6'>
            <img src={man} alt="" className='h-10' />
            <h2 className='mt-2 ml-4 font-bold'>Chat Box</h2>
            <button type="submit" className='mt-1 ml-auto mx-4'>
              <img src={out} alt="" className='h-5' />
            </button>
          </div>
        </form>
        <div className='flex justify-center mb-3 '>
          <input className='input-box rounded-3xl w-80 px-5 py-2 '
            type="search"
            placeholder="Search..."
          />
        </div>
      </div>
      <div>
        <h2 className='mx-6 text-xl font-bold text-blue-500 my-3'>Chats...</h2>
        {chatData.map((chat, index) => (
          <div key={index} className='flex border border-gray-300 p-4'>
            <img src={man} alt="" className='h-9' />
            <h2 className='mt-2 ml-4 font-bold'>{chat.username}</h2>
          </div>
        ))}
      </div>
    </div>
    <div className='col-span-3'>
      <div className='py-4 px-4 border border-gray-300 '>

```

```

<div className='flex'>
<img src={man} alt="" className='h-12' />
<h2 className='mt-2 ml-4 font-bold'>Krithee</h2>
<img src={logo} alt="" className='h-12 ml-auto' />
</div>
</div>
<div>
<div className='flex'>
<div className='relative w-full m-1'>
<input className='message w-full rounded-md pl-10 py-4'
type="text"
placeholder="Type a message"
value={message}
onChange={(e) => setMessage(e.target.value)}
/>
<button onClick={sendMessage} className="absolute inset-y-0 right-0 flex items-center">
<img src={send} alt="" className='send h-7 mx-3 mb-6' />
</button>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
);
}
export default Home

```

ChatItem.jsx

```

import React from 'react';
import man from '../components/images/man.png';

const ChatItem = ({ chat }) => (
<div className='flex border border-gray-300 p-4'>
<img src={man} alt="" className='h-9' />
<h2 className='mt-2 ml-4 font-bold'>{chat.username}</h2>
</div>
);

export default ChatItem

```

Signup.jsx

```
import React from 'react'
import { Link, useNavigate } from 'react-router-dom';
import logo from '../components/images/logo.png'

function Signup() {
  const navigate = useNavigate();
  const handleSubmit = async (event) => {
    event.preventDefault();

    const formData = {
      name: event.target.name.value,
      email: event.target.email.value,
      password: event.target.password.value,
    };
    try {
      const response = await fetch('http://localhost:5002/api/signup', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(formData),
      });
      const result = await response.json();
      console.log(result);
      if (result.success) {
        navigate('/');
      } else {
        console.error('Signup failed:', result.error);
      }
    } catch (error) {
      console.error('Error during signup:', error);
    }
  };

  return (
    <div>
      <div className='h-screen flex items-center justify-center'>
        <div className='ml-auto hidden lg:block'>
          <h1 className='text-5xl font-bold my-2 '>Share <span className='text-green-500'>Message</span></h1>
          <h3 className='text-5xl font-bold '>With ChatBox</h3>
          <img src={logo} alt="" className='my-4 h-16' />
        </div>
        <form className='flex flex-col justify-center border shadow-xl rounded-md p-9 lg:ml-auto lg:mr-48'
          onSubmit={handleSubmit}>
          <input className='"input-box text-neutral-700 text-base font-normal w-72 py-2"
            placeholder='Full Name'
```

```

type="text"
id="name"
name="name"
required
autoComplete="name"
/>
<input className="input-box text-neutral-700 text-base font-normal w-72 py-2"
placeholder='Email Address'
type="email"
id="email"
name="email"
required
autoComplete="email"
/>
<input className="input-box text-neutral-700 text-base font-normal w-72 py-2"
placeholder='Password'
type="password"
id="password"
name="password"
required
autoComplete="password"
/>
<input className="input-box text-neutral-700 text-base font-normal w-72 py-2"
placeholder='Confirm Password'
type="password"
id="Confirmpassword"
name="Confirmpassword"
required
autoComplete="Confirmpassword"
/>
<div className="signup-button w-72 h-11 rounded-lg text-center leading-10 mt-10 mb-8 bg-blue-600">
<button type="submit" className="w-36 text-white font-extrabold ">Sign Up</button>
</div>
<div className='ml-auto'><span className="text-black text-sm font-bold">Already a user ?
</span><Link to="/" className="text-blue-600 text-sm font-bold"> Sign In</Link></div>
</form>
</div>
</div>
)
}
export default Signup

```

Signin.jsx

```
import React from 'react';
import { Link, useNavigate } from 'react-router-dom';
import logo from '../components/images/logo.png';

function Signin() {
  const navigate = useNavigate();
  const handleSubmit = (event) => {
    event.preventDefault();
    navigate('/Home');
  };
  return (
    <div>
      <div className='h-screen flex items-center justify-center'>
        <div className='ml-auto hidden lg:block'>
          <h1 className='text-5xl font-bold my-2 '>Share <span className='text-green-500'>Message</span></h1>
          <h3 className='text-5xl font-bold '>With ChatBox</h3>
          <img src={logo} alt="" className='my-4 h-16' />
        </div>
        <form
          className='flex flex-col justify-center border shadow-xl rounded-md p-9 lg:ml-auto lg:mr-48'
          onSubmit={handleSubmit}
        >
          <input
            className="input-box text-neutral-700 text-base font-normal w-72 py-2"
            placeholder='Email Address'
            type="email"
            id="email"
            name="email"
            required
            autoComplete="Email"
          />
          <input
            className="input-box text-neutral-700 text-base font-normal w-72 py-2"
            placeholder='Password'
            type="password"
            id="password"
            name="password"
            required
            autoComplete="Password"
          />
          <div className="signup-button w-72 h-11 rounded-lg text-center leading-10 mt-10 mb-8 bg-blue-600">
            <button type="submit" className="w-36 text-white font-extrabold ">
              Log In
            </button>
          </div>
        </form>
      </div>
    </div>
  );
}
```

```

</div>
<div className='ml-auto'>
<span className="text-black text-sm font-bold">New User? </span>
<Link to='/Signup' className="text-blue-600 text-sm font-bold">
Sign Up
</Link>
</div>
</form>
</div>
</div>
);
}

export default Signin;

```

1.6.2 CSS: Tailwind CSS is used for styling.

Tailwind.config file

```

/** @type {import('tailwindcss').Config} */
module.exports = {
  content: [
    './src/**/*.{js,jsx,ts,tsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}

```

1.6.3 BACKEND

Package.json:

```

{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
}

```

```

"dependencies": {
  "body-parser": "^1.20.2",
  "cors": "^2.8.5",
  "express": "^4.18.2",
  "jsonwebtoken": "^9.0.2",
  "neo4j-driver": "^5.15.0",
  "nodemon": "^3.0.2",
  "socket.io": "^4.7.2"
}
}

```

Server.js

```

const express = require('express');
const cors = require('cors');
const path = require('path');
const neo4j = require('neo4j-driver');
const { auth } = neo4j;
const jwt = require('jsonwebtoken');
const bodyParser = require('body-parser');
const http = require('http');
const socketIO = require('socket.io');
const app = express();
const port = 5002;
const server = http.createServer(app);
const io = socketIO(server);

app.use(cors({
  origin: 'http://localhost:3000',
  methods: 'GET,HEAD,PUT,PATCH,POST,DELETE',
  credentials: true,
  optionsSuccessStatus: 204,
}));
app.use(bodyParser.json());
const driver = neo4j.driver('bolt://127.0.0.1:7687', auth.basic('neo4j', 'Janani2712'));
driver.onCompleted = () => console.log('Connected to Neo4j database');
driver.onError = error => console.error('Error connecting to Neo4j database', error);
const session = driver.session();
app.use(express.static(path.join(__dirname, '../chat-application/build')));
const authenticateToken = (req, res, next) => {
  const token = req.header('Authorization');
  if (!token) return res.status(401).send('Access denied. ');
  jwt.verify(token, '2712', (err, user) => {
    if (err) return res.status(403).send('Invalid token. ');
    req.user = user;
  });
  next();
}

```



```

});
};
const sendMessage = () => {
if (socket) {
socket.emit('message', { text: message }, (acknowledgment) => {
console.log('Message sent to server, acknowledgment received:', acknowledgment);
});
}
};
app.post('/api/signup', async (req, res) => {
try {
const { name, email, password } = req.body;
const result = await session.run(
'CREATE (u:User {name: $name, email: $email, password: $password}) RETURN u',
{ name, email, password }
);
res.json({ success: true, user: result.records[0].get('u').properties });
} catch (error) {
console.error(error);
res.status(500).json({ success: false, error: 'Internal Server Error' });
}
});
app.post('/api/login', async (req, res) => {
try {
const { email, password } = req.body;
const result = await session.run('MATCH (u:User {email: $email}) RETURN u', { email });
if (result.records.length === 0) {
return res.status(401).json({ success: false, error: 'Invalid credentials' });
}
const user = result.records[0].get('u').properties;
if (user.password !== password) {
return res.status(401).json({ success: false, error: 'Invalid credentials' });
}
const token = jwt.sign(user, '2712');
res.json({ success: true, token });
} catch (error) {
console.error(error);
res.status(500).json({ success: false, error: 'Internal Server Error' });
}
});
io.on('connection', (socket) => {
console.log('A user connected');
socket.on('message', (data) => {
console.log('Received message:', data);
});
socket.on('disconnect', () => {
console.log('User disconnected');
});
});

```

```

});
app.get('/api/protected', authenticateToken, (req, res) => {
res.json({ success: true, message: 'You have access to this protected route.' });
});
app.get('/api/data', async (req, res) => {
try {
const result = await session.run('MATCH (n) RETURN n LIMIT 25');
const nodes = result.records.map(record => record.get('n').properties);
res.json(nodes);
} catch (error) {
console.error(error);
res.status(500).send('Internal Server Error');
}
});
app.get('*', (req, res) => {
res.sendFile(path.join(__dirname, '../chat-application/build', 'index.html'));
});
app.listen(port, () => {
console.log(`Server is running on http://localhost:${port}`);
});

```

Socket.js

```

const socketIO = require('socket.io');
const initializeSocket = (server) => {
const io = socketIO(server);
io.on('connection', (socket) => {
console.log('User connected:', socket.id);
socket.on('message', (data) => {
console.log('Received message:', data);
});
socket.on('disconnect', () => {
console.log('User disconnected:', socket.id);
});
});
return io;
};
module.exports = initializeSocket;

```

1.6.4 NEO4J DESKTOP:

Creating a new project and running the project is shown in Fig 1.2.

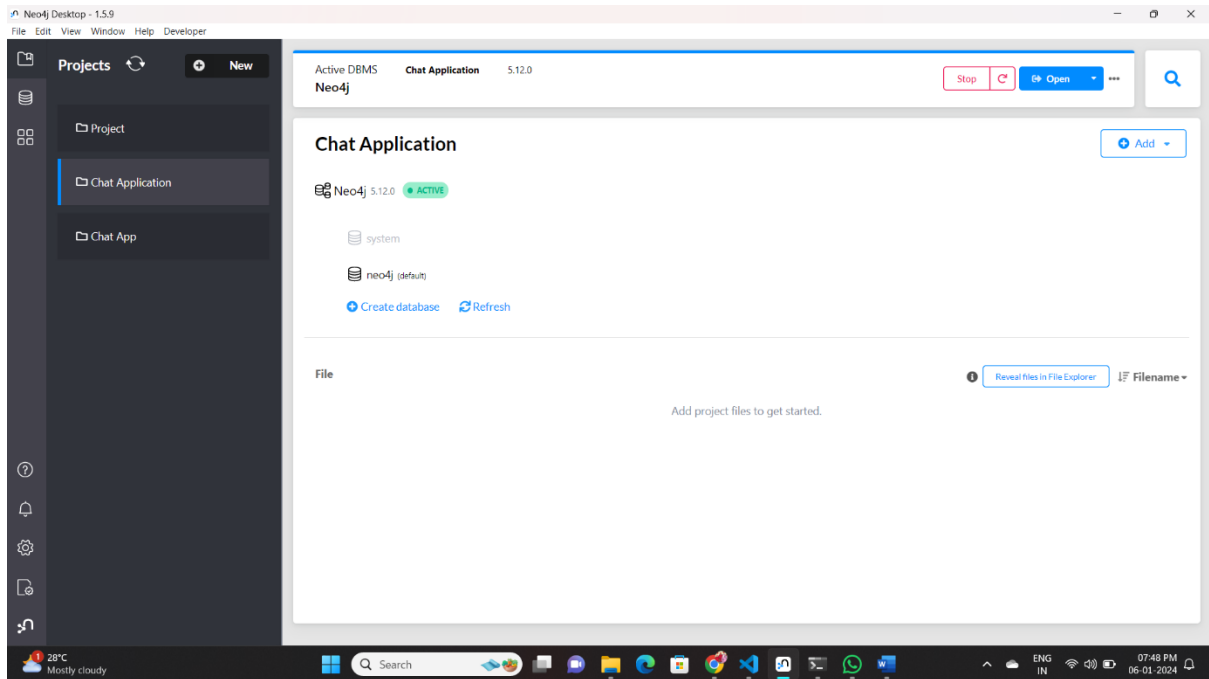


Fig 1.2 Neo4j Desktop

Server and Connection Status is shown in Fig 1.3.

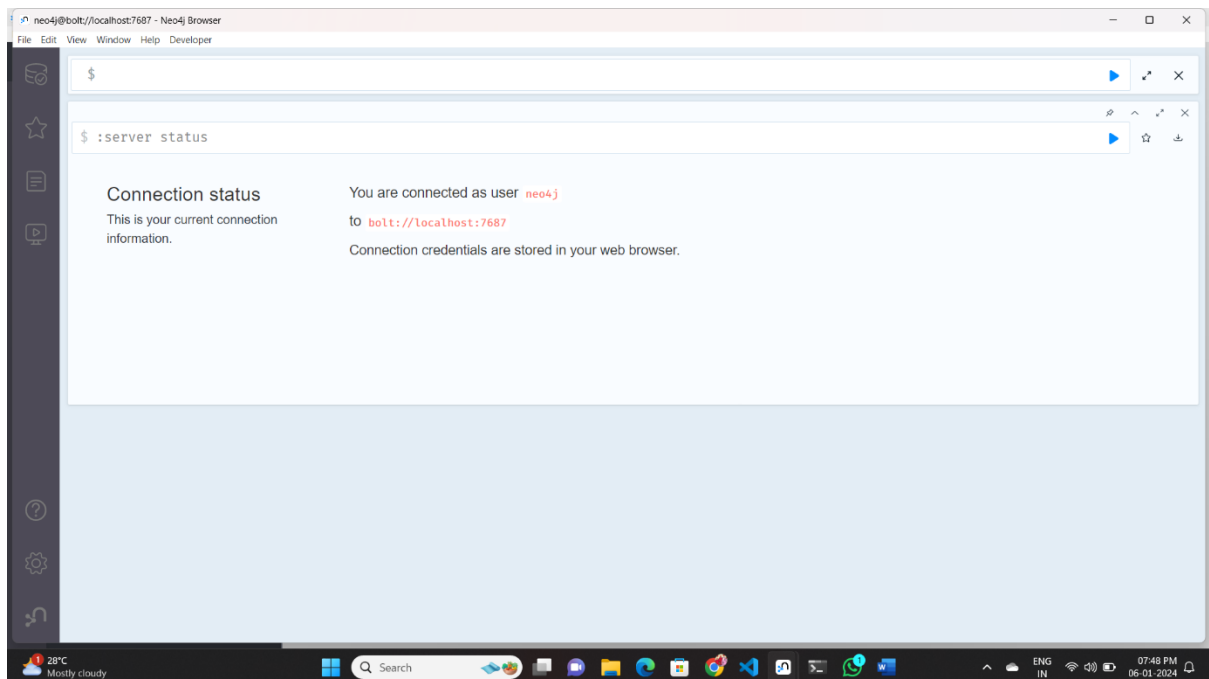


Fig 1.3 Server Status in Neo4j Desktop

CHAPTER 2

OUTPUT AND DISCUSSION

FRONTEND:

The Index page of our Chat Application is shown in Fig 2.1.

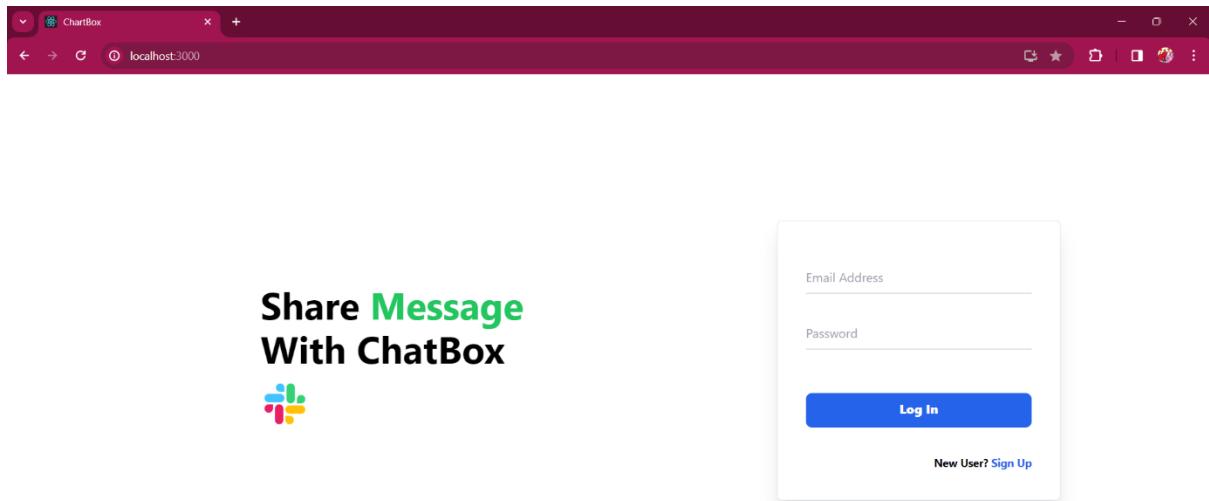


Fig 2.1 Index Page

SIGNUP:

If you are a New User, you should register yourself in Signup page is shown in Fig 2.2.

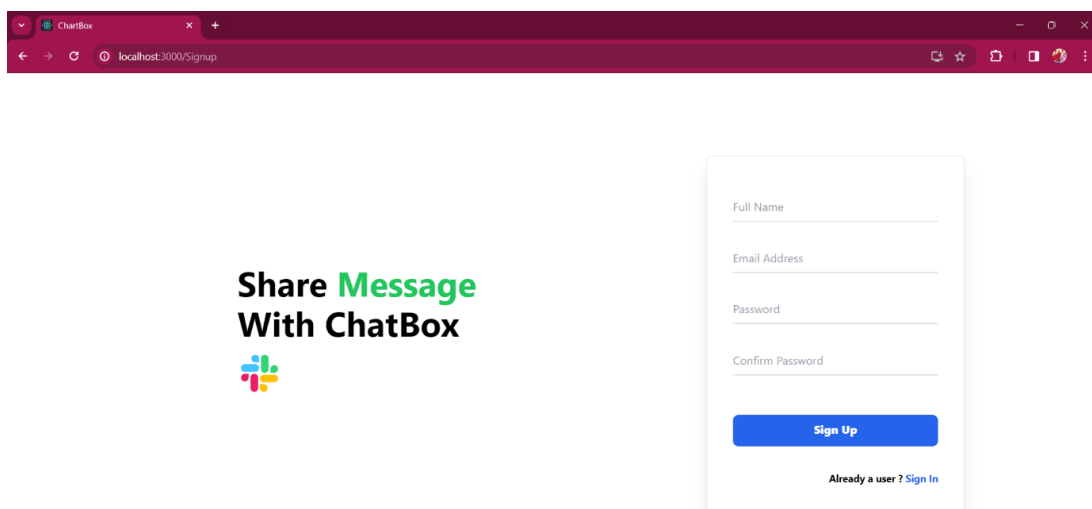


Fig 2.2 Signup Page

SIGN IN:

If you are already a User, you can directly use Sign In is shown in Fig 2.3.

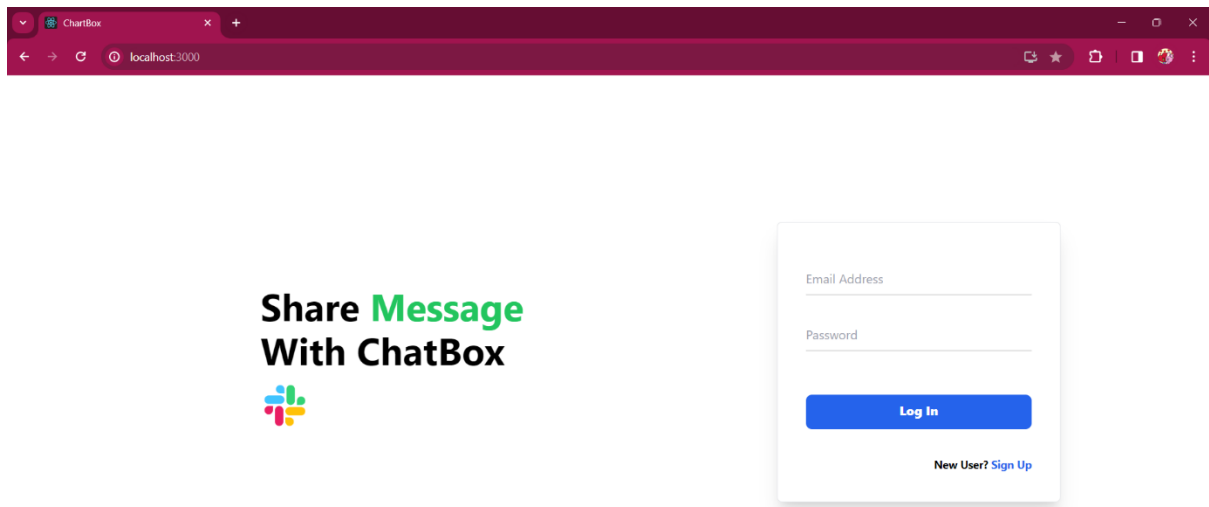


Fig 2.3 Signin Page

HOME:

The Home page where users will Chat with their Friends is shown in Fig 2.4.

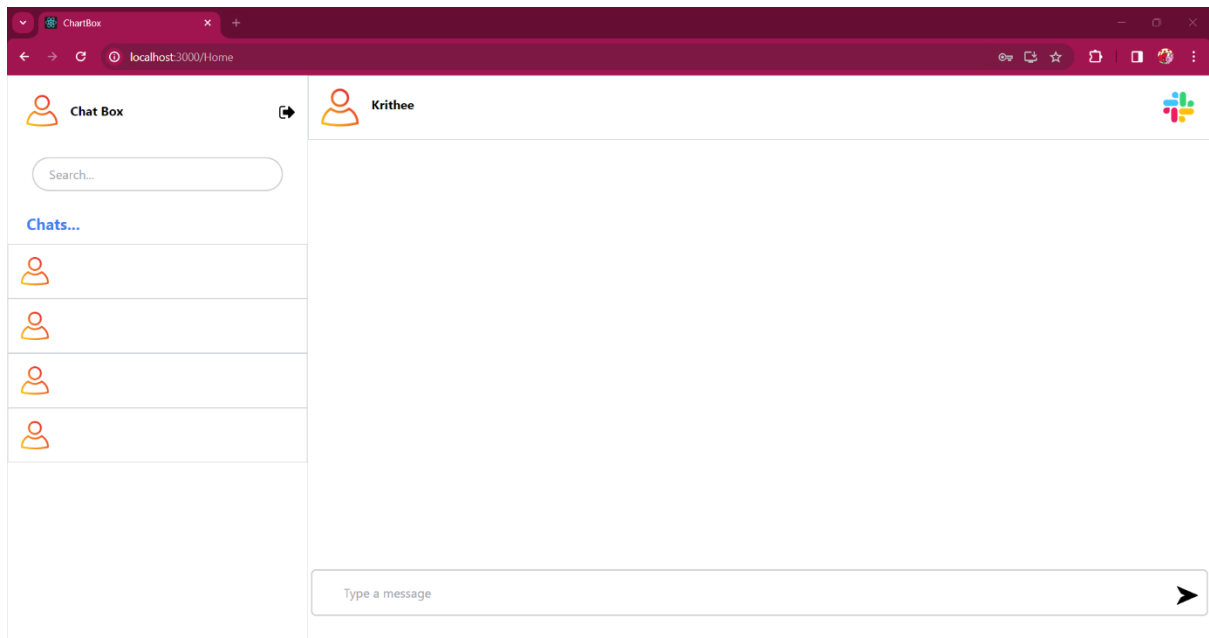


Fig 2.4 Home Page

BACKEND:

These are Four Users that are register through Frontend is shown in Fig 2.5.

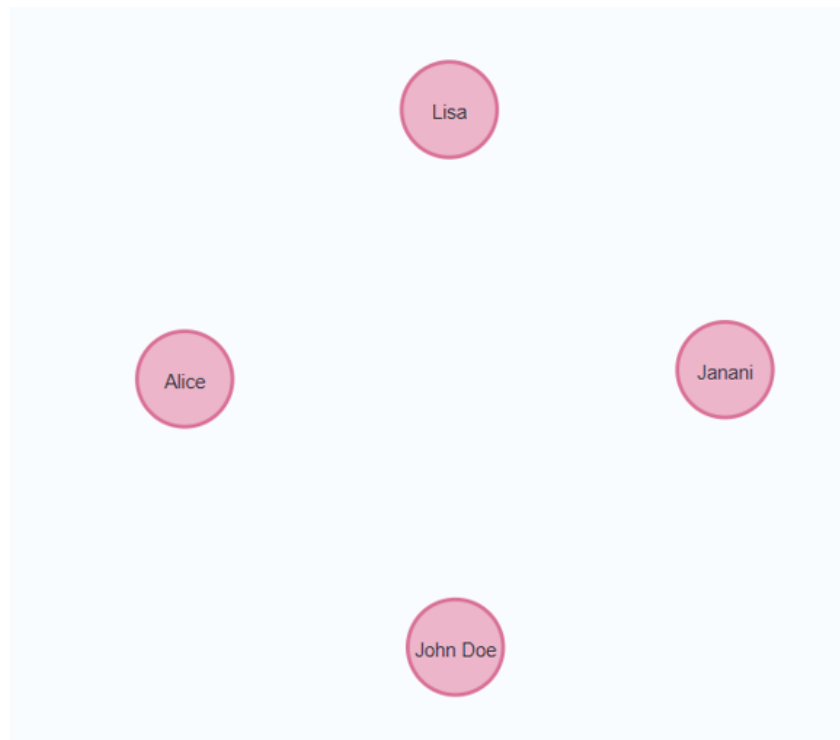


Fig 2.5 Registered Users

CREATE:

Query: MATCH (user:User {name: 'John Doe'}), (message:Message {text: 'Hello, how are you?'}) CREATE (user)-[:SENT]->(message). Creating a message is shown in Fig 2.6.

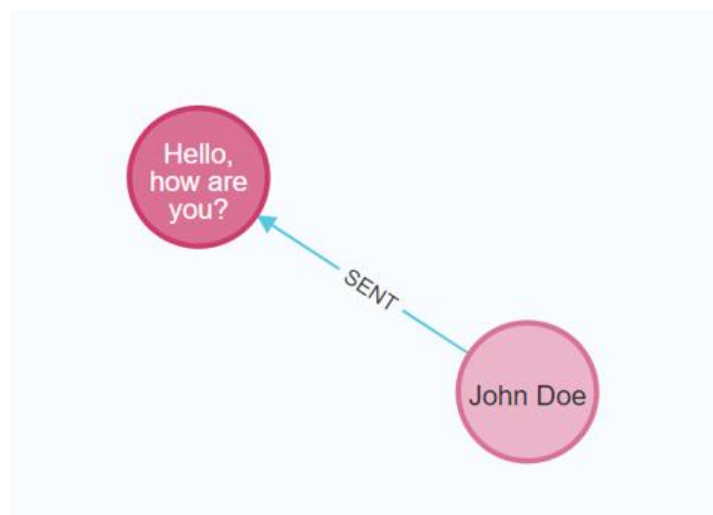


Fig 2.6 John Sent a Message

Query: MATCH (user:User {name: 'Janani'}), (message:Message {text: 'Hello, how are you?'}) CREATE (user)-[:RECEIVED]->(message).Receiving a Message is shown in Fig 2.7.

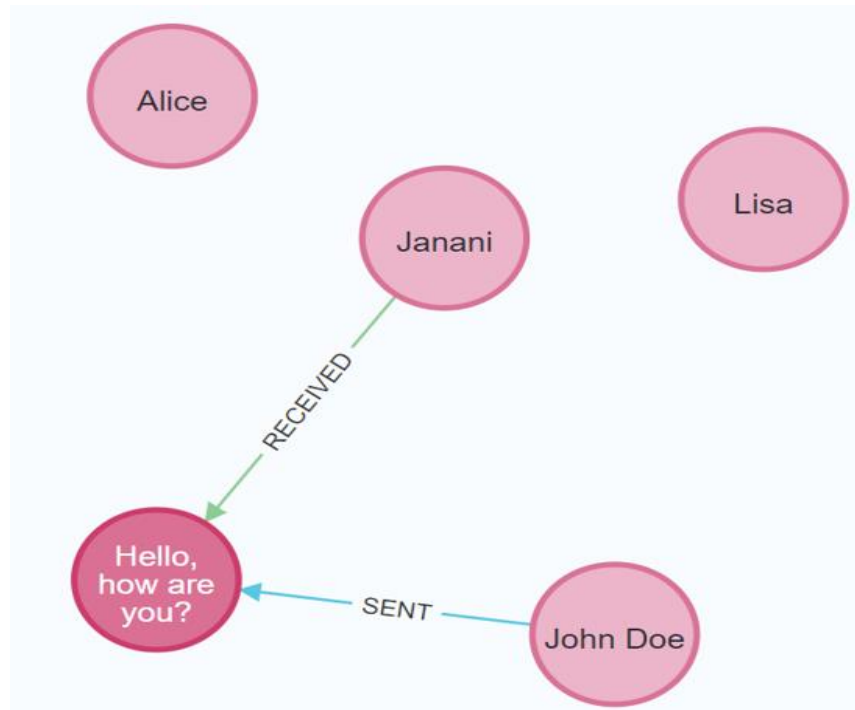


Fig 2.7 Janani Received a Message

Query: MATCH (sender:User {name: 'Alice'}), (receiver:User {name: 'Lisa'})
CREATE (message:Message {text: 'Hey Lisa, how are you?'})
CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.8.

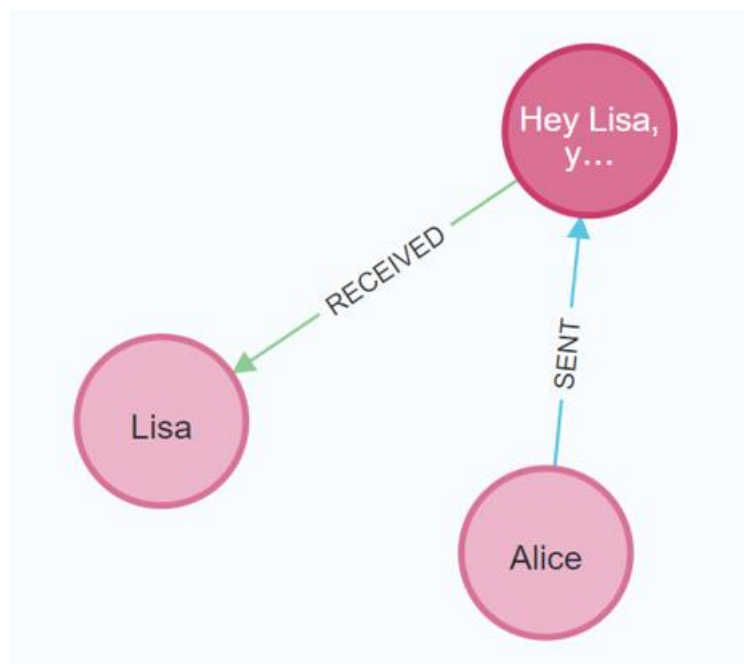


Fig 2.8 Alice Sent Message to Lisa

Query: MATCH (sender:User {name: 'Lisa'}), (receiver:User {name: 'John Doe'})
 CREATE (message:Message {text: 'Hi John, what are you up to?'})
 CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.9.

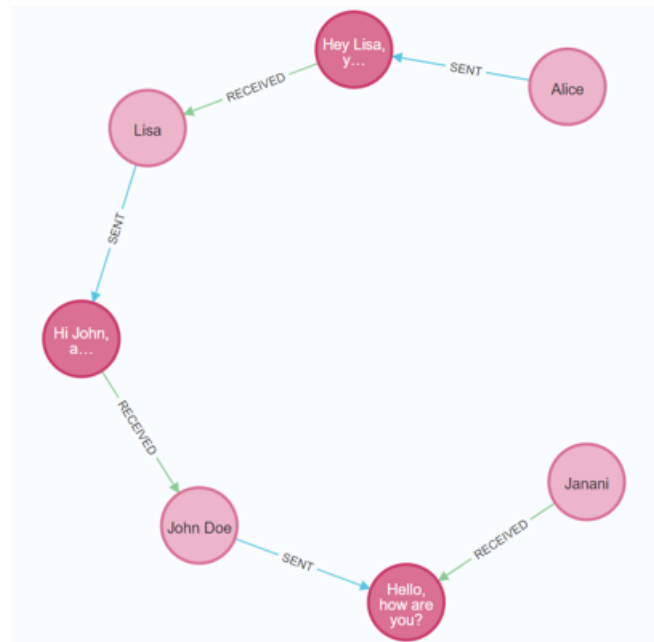


Fig 2.9 Lisa sent a message to John Doe

Query: MATCH (sender:User {name: 'Janani'}), (receiver:User {name: 'Lisa'})
 CREATE (message:Message {text: 'Hello Lisa, how's it going?'})
 CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.10.

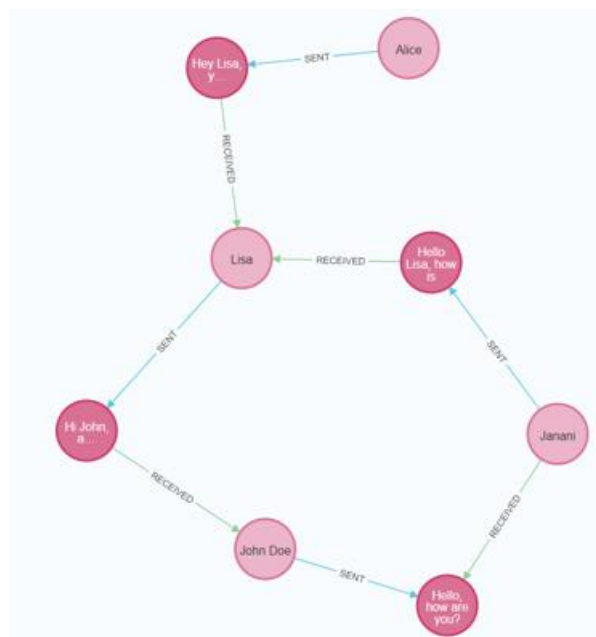


Fig 2.10 Janani sent a message to Lisa

Query: MATCH (sender:User {name: 'Janani'}), (receiver:User {name: 'John Doe'})
 CREATE (message:Message {text: 'Hi John, long time no talk!'})
 CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.11.

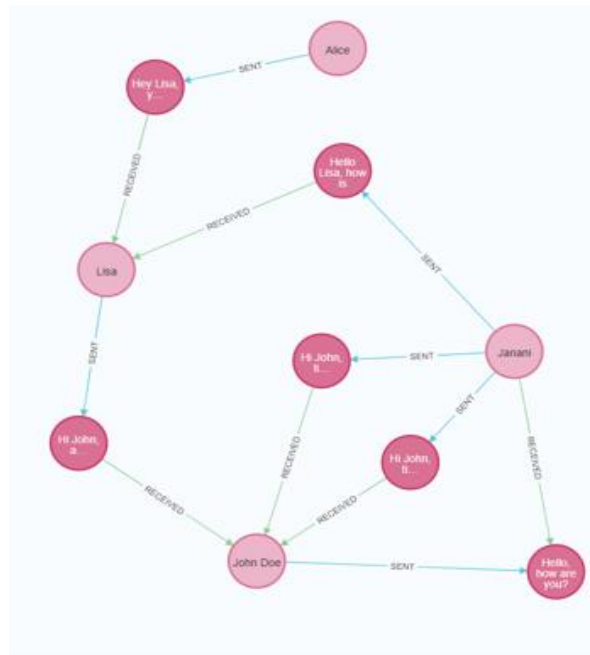


Fig 2.11 Janani sent a message to John Doe

Query: MATCH (sender:User {name: 'Janani'}), (receiver:User {name: 'Alice'})
 CREATE (message:Message {text: 'Hi Alice, how is your work?'})
 CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.12.

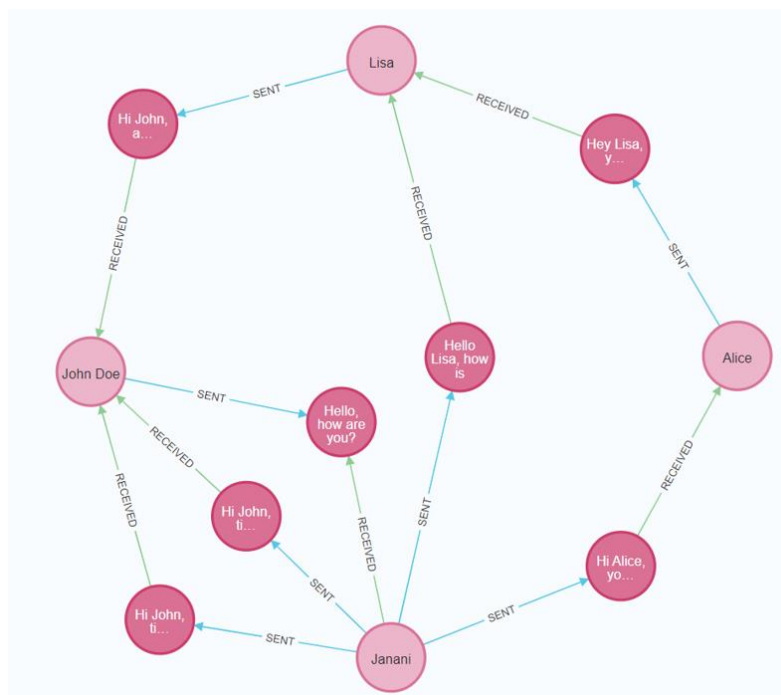


Fig 2.12 Janani sent a message to Alice

Query: MATCH (sender:User {name: 'Lisa'}), (receiver:User {name: 'Alice'})

CREATE (message:Message {text: 'Hey Alice, what's up?'})

CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig 2.13.

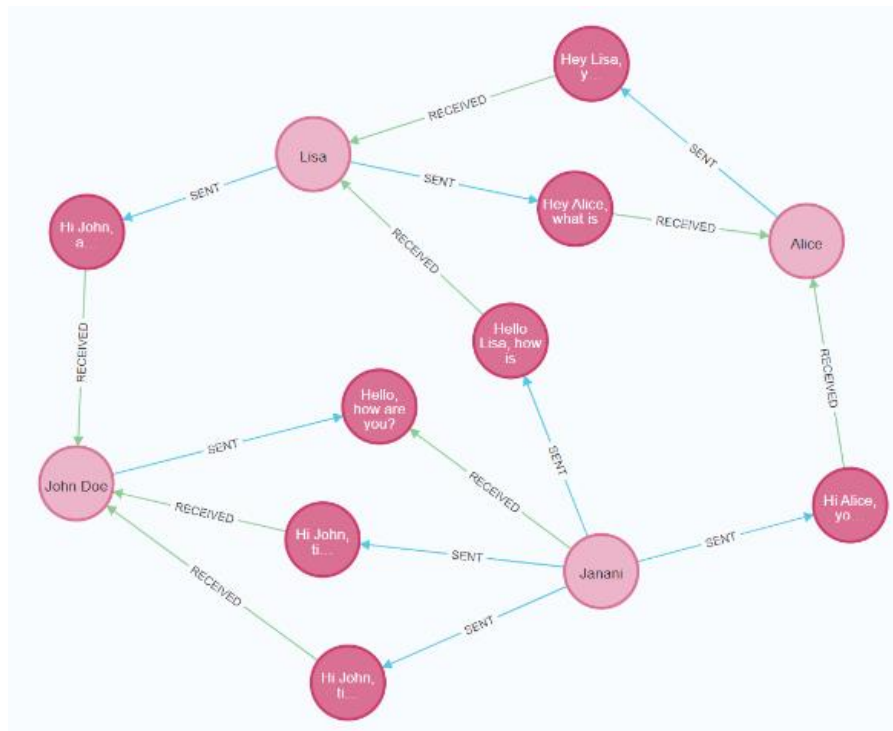


Fig 2.13 Lisa sent a message to Alice

Query: MATCH (sender:User {name: 'Alice'}), (receiver:User {name: 'Janani'}) CREATE (message:Message {text: 'Hello Janani, nice to meet you!. My Work is going good'})

CREATE (sender)-[:SENT]->(message)-[:RECEIVED]->(receiver) is shown in Fig2.14.

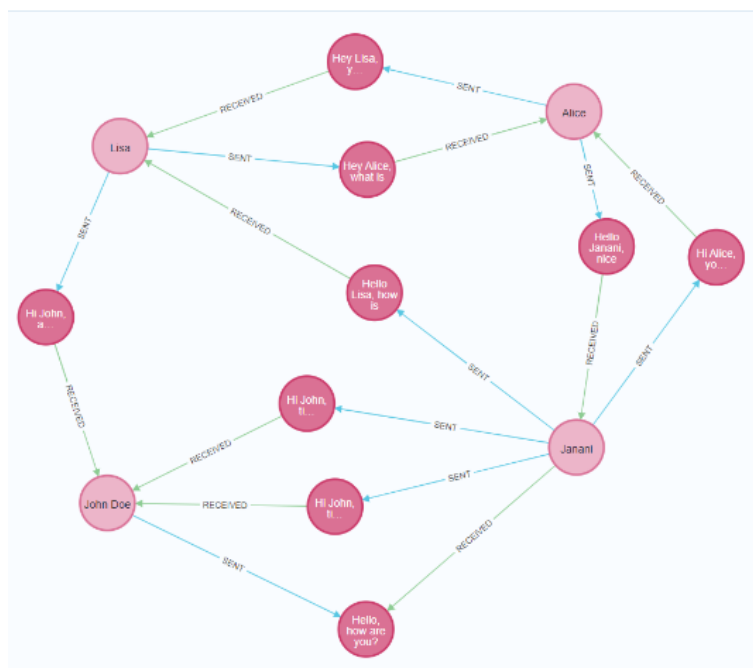


Fig 2.14 Alice sent a message to Janani

READ:

To see all the Users is shown in Fig 2.15.

Query: MATCH (user:User) RETURN user

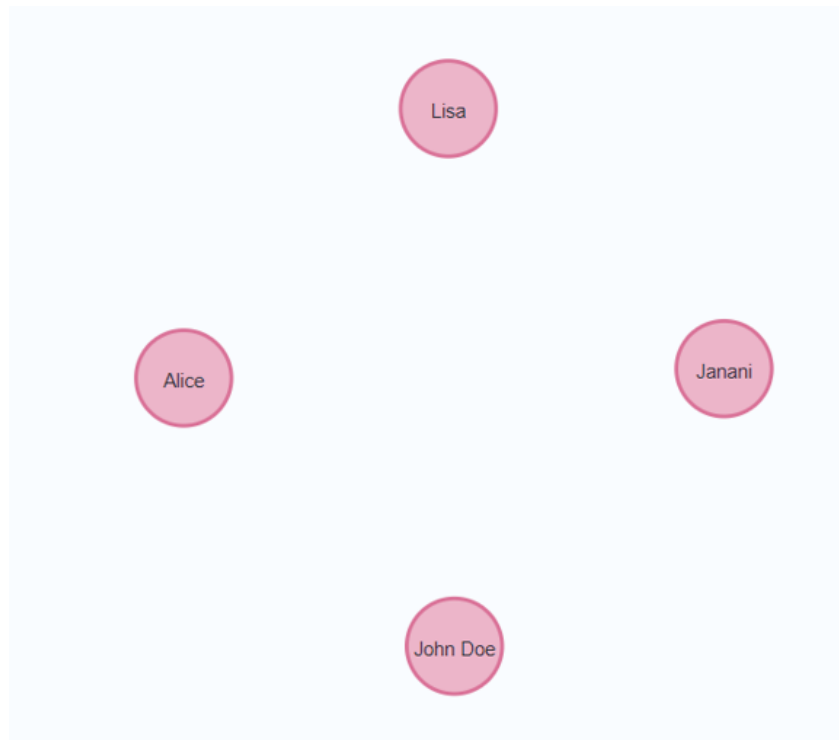


Fig 2.15 All User's

To see all the Messages is shown in Fig 2.16.

Query: MATCH (message:Message) RETURN message

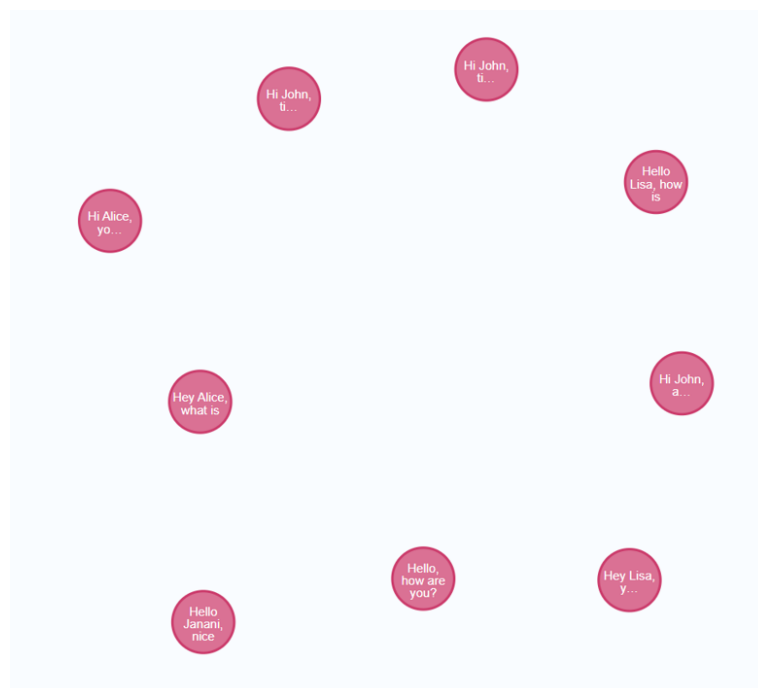


Fig 2.16 All Messages

UPDATE:

Here, I am updating a user's email is shown in Fig 2.17.

Query: MATCH (alice:User {name: 'Alice'}) SET
alice.email='alicewonderland@gmail.com' RETURN alice .

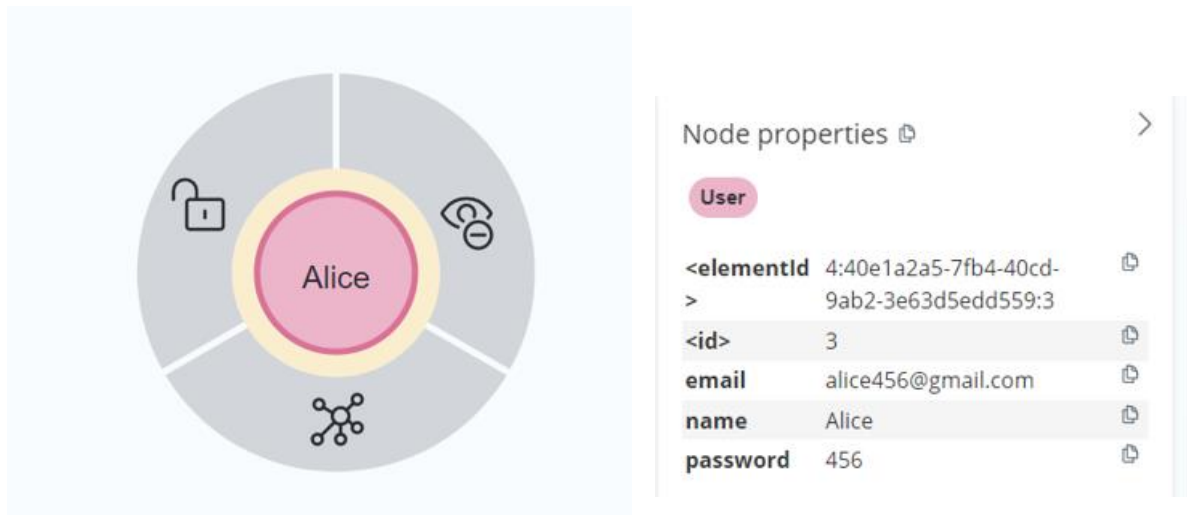


Fig 2.17 Updated User's Email

COUNT:

How many Users are there is shown in Fig 2.18.

Query: MATCH (user:User) RETURN COUNT(user) AS userCount

```
neo4j$ MATCH (user:User) RETURN COUNT(user) AS userCount
```

	userCount
1	4

Fig 2.18 User's Count

GROUP AND COUNT:

Query: MATCH (sender:User)-[:SENT]->(message:Message) RETURN sender.name AS sender, COUNT(message) AS sentMessages is shown in Fig 2.19.

	sender	sentMessages
1	"John Doe"	1
2	"Alice"	2
3	"Lisa"	2
4	"Janani"	4

Fig 2.19 Group and Count

AGGREGATION WITH FILTERING:

Query: MATCH (user:User)-[:RECEIVED]->(message:Message) WITH user, COUNT(message) AS receivedMessages ORDER BY receivedMessages DESC LIMIT 1 RETURN user.name AS userWithMostReceivedMessages, receivedMessages

	userWithMostReceivedMessages	receivedMessages
1	"Janani"	1

Fig 2.20 Aggregation with Filtering

DELETE:

First I am creating a new user and the user sending a text to their Friend is shown in Fig 2.21.

Query: CREATE (newUser:User {name: 'Bob', password: '789', email: 'bob123@gmail.com'}) WITH newUser MATCH (recipient:User {name: 'Lisa'})

CREATE (newUser)-[:SENT]->(newMessage:Message {text: 'Hi Lisa, this is Bob!')-[:RECEIVED]->(recipient)

Query: MATCH (sender:User {name: 'Bob'})-[:SENT]->(message:Message)-[:RECEIVED]->(recipient:User {name: 'Lisa'}) DETACH DELETE sender, message



Fig 2.23 Deleting a New User along with the Relation

After Deleting my Final Output will be shown in Fig 2.24.

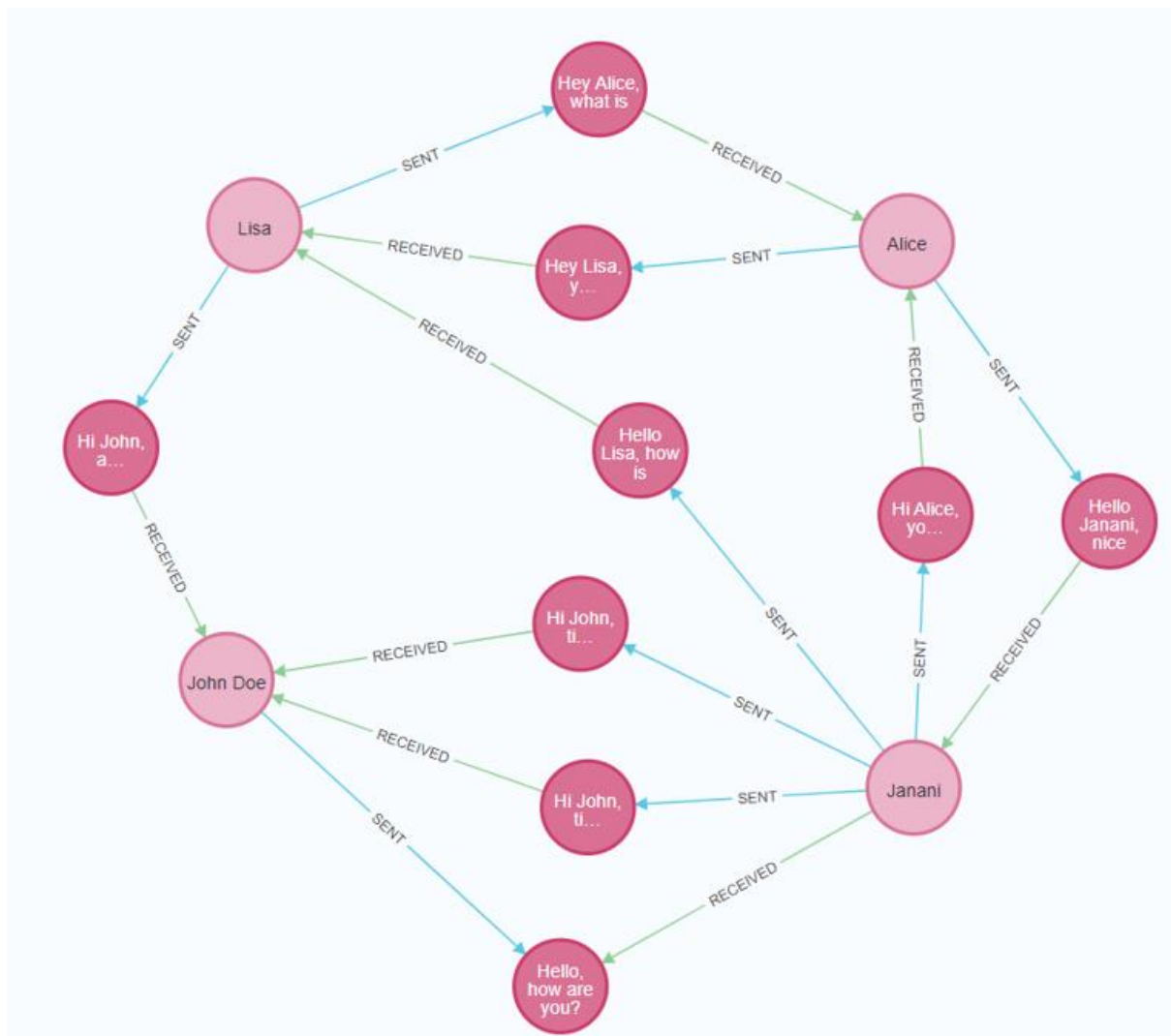


Fig 2.24 Delete Operation

CHAPTER 3

CONCLUSIONS AND FUTURE ENHANCEMENTS

3.1 CONCLUSIONS

In conclusion, the Social Chat application represents a successful amalgamation of innovative technologies, with React.js and Neo4j Database at its core. The project has achieved its objectives of providing a dynamic, user-friendly platform for real-time messaging within social circles. The collaborative efforts of our team have resulted in a robust and scalable system architecture that meets the diverse communication needs of our target audience. As we reflect on the journey of developing the Social Chat application, the challenges faced and overcome have contributed to our growth and understanding of complex, interconnected systems. The project's outcomes underscore the significance of integrating cutting-edge technologies for creating impactful and user-centric applications in the realm of digital communication.

3.2 FUTURE ENHANCEMENTS

Moving forward, the Social Chat application holds potential for refinement and expansion. Key areas for future enhancements include upgrading user experience through additional features, fortifying security measures, integrating mobile applications for wider accessibility, exploring AI-powered functionalities, advancing community and group management tools, and globalizing the application for a diverse user base. These enhancements aim to not only improve existing features but also propel the application into new realms of functionality, ensuring its continued relevance in the evolving landscape of digital communication.

REFERENCES

- [1]. Ian Robinson, Jim Webber, Emil Eifrem, “Graph Databases”, O’Reilly Media, 2nd Edition, 2015.
- [2]. React.js Documentation: <https://react.dev/>
- [3]. Neo4j Documentation: <https://neo4j.com/>
- [4]. Node.js Documentation: <https://nodejs.org/docs/latest/api/>
- [5]. WebSocket Documentation:
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [6]. JSON Web Token (JWT) Documentation: <https://jwt.io/>
- [7]. Material-UI Documentation: <https://mui.com/material-ui/>